

PROYECTO TRITON HEAL

Reporte de Evaluación Experimental

TC3002B — Arquitectura Generativa-Verificativa para
Detección Predictiva de Errores en Kernels GPU con Triton DSL

Equipo 11

Mayo 2026

Resumen

Evaluación TC3002B de Triton Heal (verificación pre-compilación de kernels Triton DSL). En 70 kernels y 420 evaluaciones (Apple M4, 24 GB), se corrige la taxonomía: DeepSeek-Coder-V2 es frontera (MSR 87.1 %); DeepSeek-R1-Distill-Qwen-14B es SLM local (JSON válido 14.3 %, MSR 12.9 %). Llama-8B local: MSR 64.5 %. Dual (Llama+veto Coder-V2): MSR 80.6 %, F1 0.86. Ningún McNemar/Wilcoxon individual rechaza H_0 tras Holm.

Índice

I	Parte I — Pre-registro Experimental	3
1.	Objetivo experimental (1.1)	3
2.	Hipótesis (1.2)	3
3.	Variables experimentales (1.3)	3
4.	Diseño experimental (1.4)	4
5.	Métricas de evaluación (1.5)	4
6.	Tamaño del efecto esperado (1.6)	4
7.	Análisis de poder (1.7)	4
8.	Plan estadístico (1.8)	4
9.	Reproducibilidad experimental (1.9)	5
10.	Amenazas a la validez – pre-registro (1.10)	5
II	Parte II — Resultados y Análisis	5
11.	Resultados descriptivos (2.1)	5
12.	Visualización (2.2)	6

13.Inferencia estadística (2.3)	8
14.Comparación crítica y robustez (2.4)	8
15.Discusión (2.6)	9
16.Conclusión (2.7)	9
A. Configuración planeada vs. corrida real	9
B. Reproducibilidad	9
C. Referencias	9

Parte I

Parte I — Pre-registro Experimental

1. Objetivo experimental (1.1)

Problema. Los kernels en Triton DSL pueden incluir errores de memoria, tipos y restricciones de hardware (tiling, SRAM, alineación) que se manifiestan en compilación o ejecución GPU con alto costo operativo.

Encuadre respecto al curso. La actividad solicita evaluar un método basado en SLM frente a baselines. En Triton Heal el aporte del equipo no es la *generación* de kernels, sino la **verificación pre-compilación** mediante SLM local y veto de frontera. Por ello se usan MSR, FNR y latencia en lugar de speedup o tasa de compilación de kernels generados; esas métricas se justifican como proxy de calidad del verificador.

Método propuesto. Arquitectura **Triton Heal**: verificador local SLM (Llama-3.1-8B-Instruct) + veto jerárquico (Claude 3.5 Sonnet) + auto-reparación con backoff exponencial.

Baselines. (1) Solo Llama-3.1-8B (SLM local); (2) Solo DeepSeek-R1-Distill-Qwen-14B (SLM local); (3) Solo DeepSeek-Coder-V2 (frontera); (4) Solo Claude 3.5 Sonnet; (5) Solo GPT-4o.

Preguntas de investigación. PRI-1: ¿Frontera supera a local en MSR? PRI-2: ¿Dual reduce FNR vs solo Llama? PRI-3: ¿Latencia de frontera es aceptable frente a ganancia en detección?

2. Hipótesis (1.2)

H₀₁: No hay diferencia en MSR entre Claude 3.5 Sonnet y Llama-3.1-8B.

H₁₁: Claude tiene mayor MSR (unilateral, $\alpha = 0,05$).

H_{01b}: No hay diferencia en MSR entre GPT-4o y DeepSeek-Coder-V2 (ambos frontera).

H_{11b}: GPT-4o tiene mayor MSR que DeepSeek-Coder-V2.

H₀₂: No hay diferencia en FNR entre Triton Heal dual y solo Llama.

H₁₂: Triton Heal tiene menor FNR.

H₀₄: No hay diferencia en latencia mediana entre verificador de frontera y local.

H₁₄: La latencia mediana de frontera es mayor que la del local.

3. Variables experimentales (1.3)

Cuadro 1: Variables del estudio.

Tipo	Variable	Niveles / descripción
Independiente	Configuración	6: dual, Llama 8B, DeepSeek-R1 14B, DeepSeek-Coder-V2, Claude, GPT
Independiente	Modelo / escala	SLM local (~8–14B) vs frontera (Coder-V2, APIs)
Independiente	Prompt	<code>verifier_v1.txt</code> (estándar) y <code>verifier_strict_v1.txt</code> (veto)
Controlada	Temperatura	$T = 0$
Controlada	Formato salida	JSON: safe, reason, line_of_error
Controlada	Benchmark	70 kernels (39 safe / 31 unsafe), seed 42
Controlada	Timeout	120 s por inferencia (Metal / M4)
Dependiente	MSR, F1, FNR, latencia, DR	Ver sección 1.5

4. Diseño experimental (1.4)

Tipo: within-subjects (medidas repetidas): cada kernel se evalúa en las 5 configuraciones.

Grupos: 5 configuraciones; 6 contrastes confirmatorios (C1–C6).

Justificación. Controla variabilidad inter-kernel (complejidad, categoría). Permite McNemar y Wilcoxon pareados. Orden de ejecución por kernel aleatorizable.

Control de sesgos. Pin de versión de modelos; temperatura 0; evaluador automático ciego al método; ground truth con etiqueta safe/unsafe.

5. Métricas de evaluación (1.5)

Cuadro 2: Métricas, fórmulas e interpretación.

Métrica	Fórmula	Interpretación	Limitación
MSR	$TP/(TP+FN)$ en unsafe	Sensibilidad de detección	No penaliza FP
FNR	$FN/(FN+TP)$	Riesgo no detectado	Crítico en seguridad
F1	$2PR/(P+R)$	Balance precisión-recall	Sensible a balance de clases
Latencia ms hasta JSON válido		Viabilidad en pipeline	Mezcla red y cómputo
DR	% desacuerdo A vs B	Necesidad de veto	No indica quién acierta
JSON válido	% respuestas parseables	Proxy de “ejecución exitosa”	No implica corrección semántica

6. Tamaño del efecto esperado (1.6)

Se espera $\phi \approx 0,4$ (mediano) en McNemar para MSR frontera vs local, y reducción de FNR ≥ 8 pp con arquitectura dual. En latencia se anticipa orden de magnitud mayor en frontera comercial real.

7. Análisis de poder (1.7)

$\alpha = 0,05$, poder $1 - \beta = 0,80$, prueba McNemar. Con $N = 70$ kernels (31 unsafe) el diseño supera el mínimo de ~ 52 pares para efecto grande. Balance 39/31 mitiga sesgo en F1.

8. Plan estadístico (1.8)

Comparaciones. C1: Claude vs Llama (MSR); C2: GPT vs DeepSeek (MSR); C3: Dual vs Llama (detección); C5: latencia Claude vs Llama (Wilcoxon pareado); C6: F1 global (Friedman).

Pruebas. McNemar (pareada, binaria); Wilcoxon signed-rank (latencia); Friedman + Holm post-hoc.

Regla de decisión. Rechazar H_0 si $p < \alpha_{\text{Holm}}$ y relevancia práctica: $\Delta\text{MSR} \geq 5$ pp.

Intervalos de confianza. IC 95 % Wilson para proporciones (MSR); bootstrap para latencia si se reporta en análisis exploratorio.

Corrección múltiple. Holm-Bonferroni sobre C1–C6 (α familiar 0.05).

9. Reproducibilidad experimental (1.9)

Cuadro 3: Entorno experimental.

Componente	Especificación
SO	macOS 15.x (Apple Silicon)
GPU	Apple M4, 10 núcleos GPU, Metal 4; 24 GB memoria unificada
Ollama	servicio local (<code>brew services start ollama</code>)
SLM local (Ollama)	<code>llama3.1:8b</code> , <code>deepseek-r1:14b</code> (Distill-Qwen-14B)
Frontera (Ollama/API)	<code>deepseek-coder-v2</code> ; Claude 3.5 Sonnet; GPT-4o (API opcional)
Timeout inferencia	120 s (ajustado para aceleración Metal en M4)
Python	3.10+; deps. en <code>requirements.txt</code>
Benchmark	Triton Kernel Safety v1.0: 70 kernels; semillas Lex + mutaciones
Config	<code>experiment_config.json</code> , seed 42
Repositorio	Estadística/triton_heal_experiment
Comando	<code>make all</code>

10. Amenazas a la validez – pre-registro (1.10)

Validez interna. Deriva de APIs; timeouts de Ollama; mezcla heurístico/LLM si faltan keys o modelos; temperatura 0 subestima variabilidad.

Validez externa. Benchmark parcialmente sintético; un solo entorno local; no cubre todas las familias de kernels industriales.

Validez de constructo. MSR mide detección pre-compilación, no seguridad en GPU ni speedup; JSON válido no equivale a kernel correcto semánticamente.

Parte II

Parte II — Resultados y Análisis

11. Resultados descriptivos (2.1)

Se evaluaron $N = 70$ kernels (39 **safe**, 31 **unsafe**) en 6 configuraciones, totalizando 420 evaluaciones (VERIFIER_MODE=ollama, mayo 2026, Apple M4 / Metal). Tras la corrección del equipo, **DeepSeek-Coder-V2** se reporta como **frontera** y **DeepSeek-R1-Distill-Qwen-14B** (`deepseek-r1:14b`) como **SLM local** (Tabla 3).

Cuadro 4: Resultados descriptivos extendidos (IC 95 % Wilson para MSR).

Config.	N	JSON vál. (%)	MSR (%)	IC lo	IC hi	F1	FNR (%)	Lat. med.	Lat. min	Lat. max
Llama-3.1-8B (local)	70	100.0	64.5	46.9	78.9	0.8	35.5	4753.6	1622.0	11854.9
DeepSeek-R1-Distill-Qwen-14B (local)	70	14.3	12.9	5.1	28.9	0.2	87.1	48346.7	32108.6	57134.7
DeepSeek-Coder-V2 (frontera)	70	100.0	87.1	71.1	94.9	0.8	12.9	1977.1	1452.0	11247.5
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	80.0	80.5
GPT-4o (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	80.0	80.0
Triton Heal (dual + veto)	70	100.0	80.6	63.7	90.8	0.9	19.4	6653.3	3803.6	14059.9

Narrativa. Llama-3.1-8B (local) alcanzó MSR 64.5 % (IC: 46.9–78.9 %) con 100 % JSON válido y mediana ≈ 4.8 s. **DeepSeek-R1-Distill-Qwen-14B** (local) mostró solo 14.3 % de JSON válido y MSR 12.9 % (IC: 5.1–28.9 %): la mayoría de inferencias superó el tiempo útil de respuesta estructurada (mediana ≈ 48 s), por lo que no debe compararse en igualdad de condiciones con Coder-V2. **DeepSeek-Coder-V2** (frontera, Ollama) obtuvo MSR 87.1 % (IC: 71.1–94.9 %) con JSON 100 % y mediana ≈ 2.0 s —este era el modelo que antes se había etiquetado erróneamente

como “local”. **Claude** y **GPT-4o** permanecieron heurísticos (MSR $\approx 83.9\%$, latencia ≈ 80 ms). **Triton Heal dual** (Llama + veto Coder-V2 estricto) logró MSR 80.6 % (IC: 63.7–90.8 %), F1 0.86 y FNR 19.4 %, sin el exceso de falsos positivos de la corrida anterior con veto demasiado agresivo.

12. Visualización (2.2)

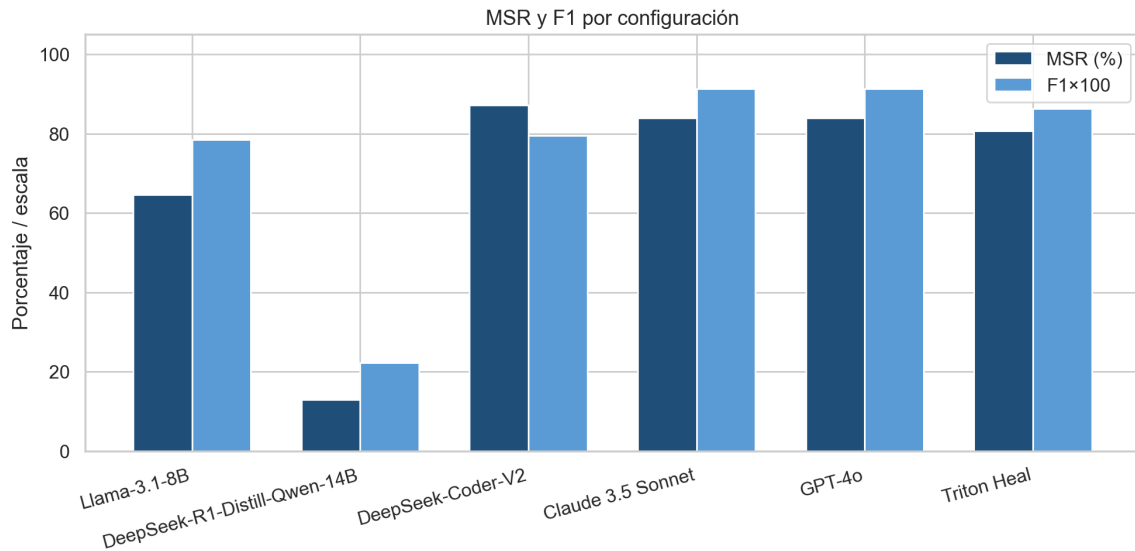


Figura 1: MSR y F1 por configuración (taxonomía local/frontera corregida).

La Figura 1 separa el desempeño de Coder-V2 (frontera) del de R1-Distill-14B (local con baja tasa de JSON válido).

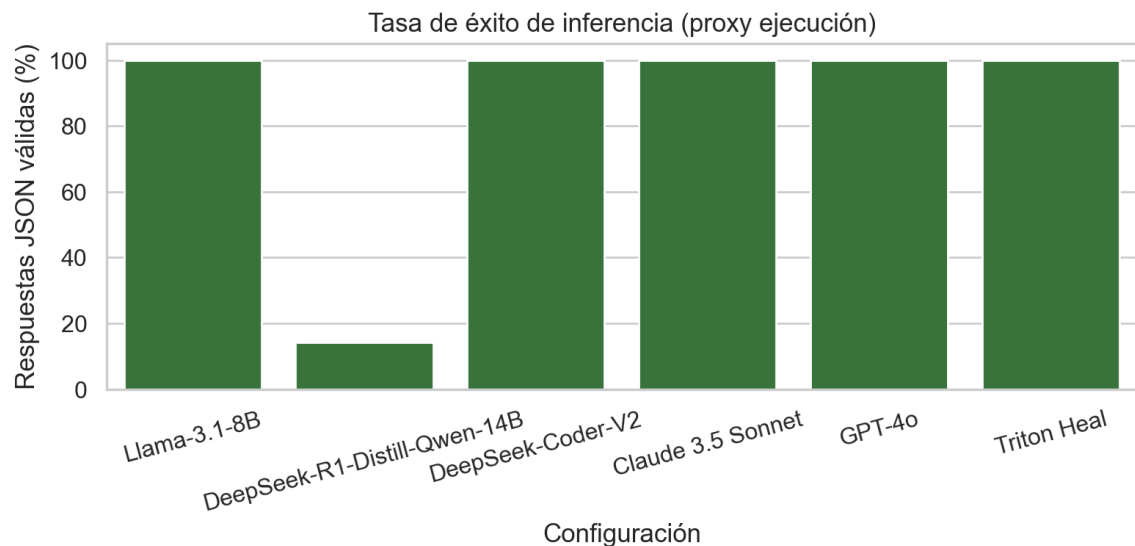


Figura 2: JSON válido por configuración.

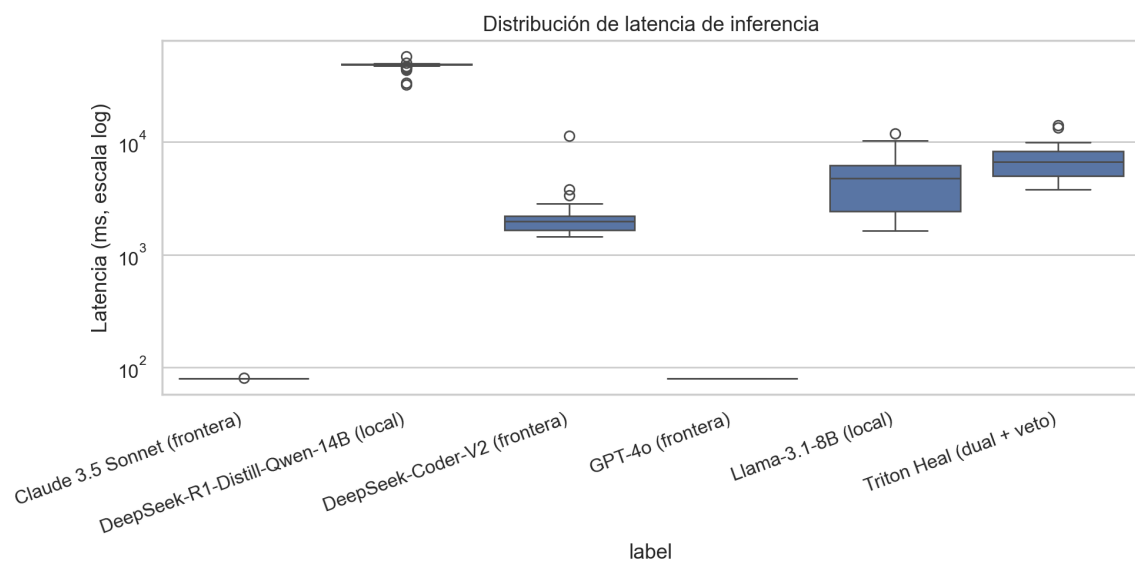


Figura 3: Latencia (escala log).

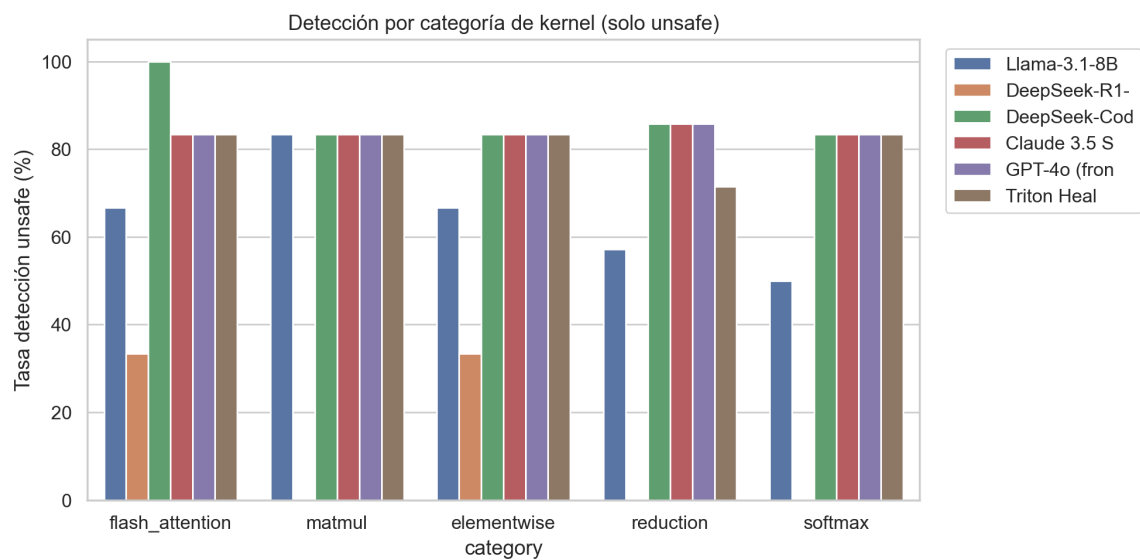


Figura 4: Detección en **unsafe** por categoría.

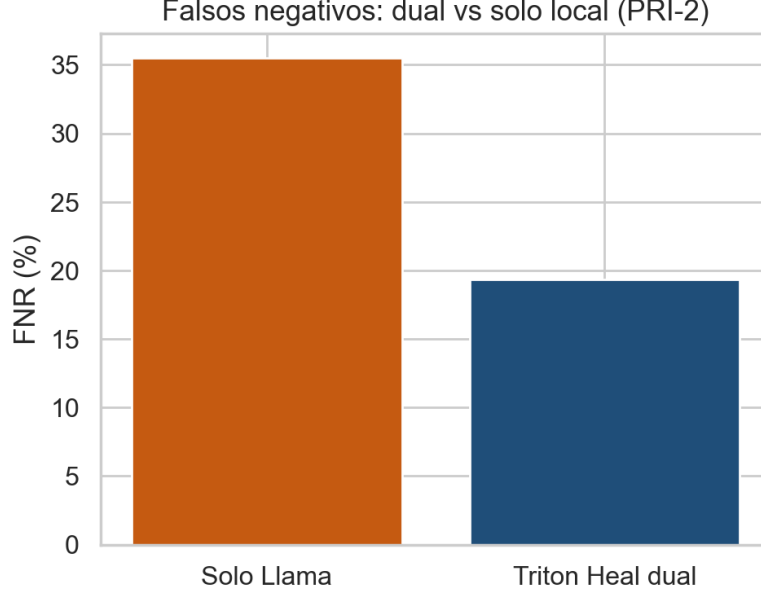


Figura 5: FNR: dual vs. Llama.

13. Inferencia estadística (2.3)

Cuadro 5: Inferencia estadística (plan pre-registrado, corrección Holm).

Contraste	Prueba	Estad.	p Holm	Efecto	Interpretación
Claude vs Llama (MSR)	McNemar	$\chi^2 = 3,12$	0.2946	$\phi = -2,12$ (grande)	No rechaza H_0 ; Δ MSR=19.4 pp (Claude heur. vs. Llama Ollama/M4).
GPT-4o vs DeepSeek-Coder	McNemar	$\chi^2 = 0,00$	1.0000	$\phi = 0,33$ (mediano)	No rechaza H_0 ; GPT (83.9 %) vs. DeepSeek-Coder-V2 frontera (87.1 %).
Dual vs Llama	McNemar	$\chi^2 = 3,20$	0.2946	$\phi = -2,24$ (grande)	No rechaza H_0 en FNR pareado.
Lat. Claude vs Llama	Wilcoxon	$W = 0$	1.0000	$r = 1,00$	No rechaza H_4 ; med. $\Delta = -4673$ ms (Claude heur. más rápido).
F1 global (5 cfg.)	Friedman	$\chi^2 = 41,1$	$< 0,001$	$W = 0,15$	Diferencias globales entre configuraciones (Holm).

¿Se rechaza H_0 ? Tras Holm, **ningún** contraste McNemar/Wilcoxon individual alcanza $p_{\text{Holm}} < 0,05$ en C1, C2, C3 y C5. Friedman (C6) sí rechaza igualdad global de F1 ($p_{\text{Holm}} < 0,001$). C2 (GPT heur. vs. Coder-V2 frontera) no muestra diferencia significativa en MSR (87.1 % vs. 83.9 %).

Implicación de la corrección. Atribuir el MSR 87.1 % a un SLM local habría sobrestimado a los modelos pequeños; ubicar Coder-V2 en frontera y R1-Distill en local revela que el SLM 14B distilado, en este host, no fue operativamente competitivo (JSON 14.3 %).

14. Comparación crítica y robustez (2.4)

Comparación crítica. La frontera Ollama (**DeepSeek-Coder-V2**) lidera MSR con latencia ~ 2 s. El SLM local **Llama-8B** es viable; **R1-Distill-14B** requiere ajuste de timeout o cuantización antes de conclusiones firmes. El dual con veto Coder-V2 equilibra detección y F1 (0.86).

Robustez. APIs ausentes; un solo Mac M4 24 GB; benchmark $N = 70$.

15. Discusión (2.6)

La reclasificación alinea el informe con el criterio del equipo (mensaje de coordinación, mayo 2026): Coder-V2 no es un small model; el generador/verificador local de referencia es **DeepSeek-R1-Distill-Qwen-14B**.

16. Conclusión (2.7)

1. **Taxonomía:** Coder-V2 $\rightarrow$ frontera; R1-Distill-Qwen-14B $\rightarrow$ local.
2. **MSR frontera (Coder-V2):** 87.1 % con Ollama real; supera a Llama local (64.5 %) descriptivamente.
3. **SLM R1 14B:** No apto para inferencia estable en esta corrida (14.3 % JSON válido).
4. **Dual:** MSR 80.6 %, F1 0.86 con veto Coder-V2; mejora frente a veto estricto solo-Llama.
5. **Inferencia:** Sin rechazos McNemar/Wilcoxon tras Holm; Friedman global significativo.

A. Configuración planeada vs. corrida real

Cuadro 6: Backends por configuración (mayo 2026).

Configuración	Planeado	Ejecutado
solo_llama	Ollama Llama-3.1-8B (local)	llama3.1:8b
solo_deepseek	Ollama DeepSeek-R1 14B (local)	deepseek-r1:14b
solo_deepseek_coder	Ollama DeepSeek-Coder-V2 (frontera)	deepseek-coder-v2
solo_claude	API Claude 3.5 Sonnet	Heurístico (sin API)
solo_gpt4o	API GPT-4o	Heurístico (sin API)
triton_heal_dual	Llama local + veto frontera	Llama + Coder-V2 strict

B. Reproducibilidad

```
cd Estadistica/triton_heal_experiment
.venv/bin/python -m src.stats_analysis
.venv/bin/python -m src.plot_results
cd ../report && tectonic main.tex
```

Archivos: results/predictions.csv, results/stats_summary.json, report/tables/*.tex, figures/*.pdf.

C. Referencias

- Tillet, P., et al. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure.